

Automated Defect Detection in Manufacturing using EfficientNet-B0: A Deep Learning Approach

Abstract

The automation of quality control processes in manufacturing industries is critical for maintaining high standards and reducing reliance on manual inspection. This study presents a machine vision-based approach utilizing a deep convolutional neural network, specifically EfficientNet-B0, for the automated classification of defective and non-defective objects. Employing a dataset of nut surfaces, the model classifies items into five fine-grained categories: Deformation, Fracture, Rusting, Scratches (Defective), and Excellent (Non-Defective). The model achieves an overall validation accuracy of 98.41% and a macro F1-score of 0.9798. This paper details the architecture, preprocessing pipeline, two-stage training methodology, and performance evaluation metrics, highlighting why EfficientNet-B0 was selected over heavier architectures such as YOLO v9.

1. Introduction

In product-based and manufacturing companies, the continuous inspection of manufactured parts is a resource-intensive and error-prone process when performed manually. The overarching goal of this project is to automate the quality control pipeline by introducing an intelligent classification system capable of differentiating between defective and non-defective objects. By automating this process, industries can ensure that no manual classification is needed, thereby accelerating production and minimizing human error.

To accomplish this, we utilize the "Nut Surface Defect Dataset" obtained from Kaggle (<https://www.kaggle.com/datasets/weihaoareal/nut-surface-defect-dataset>). The dataset provides images of nuts with various surface conditions, which we group into a broader binary classification of defective versus non-defective.

2. Model Selection and Architecture

2.1 Why EfficientNet-B0?

The selection of the underlying neural network architecture is a crucial step in ensuring both high accuracy and computational efficiency. We opted for the EfficientNet-B0 model over other object detection models like YOLO v9. While YOLO v9 is a state-of-the-art model for real-time object detection, it is inherently designed for complex bounding-box regression and multi-object tracking, which requires significantly more computational power and memory overhead. Since our primary objective is image-level classification rather than localization, YOLO v9 would be computationally excessive.

EfficientNet-B0, on the other hand, utilizes a compound scaling strategy that uniformly scales network depth, width, and resolution. As referenced in recent literature on defect quantification

(e.g., "EfficientNet-b0-Based 3D Quantification Algorithm for Rectangular Defects in Pipelines" and "Defective Photovoltaic Module Detection Using EfficientNet-B0"), EfficientNet-B0 achieves superior expressiveness with fewer parameters and Floating-Point Operations (FLOPs). It leverages Mobile Inverted Bottleneck Convolution (MBConv) and Squeeze-and-Excitation (SE) modules, making it highly suitable for industrial applications where computational resources may be limited but high-resolution feature extraction is necessary to identify micro-defects like scratches and fractures.

2.2 Model Architecture Parameters

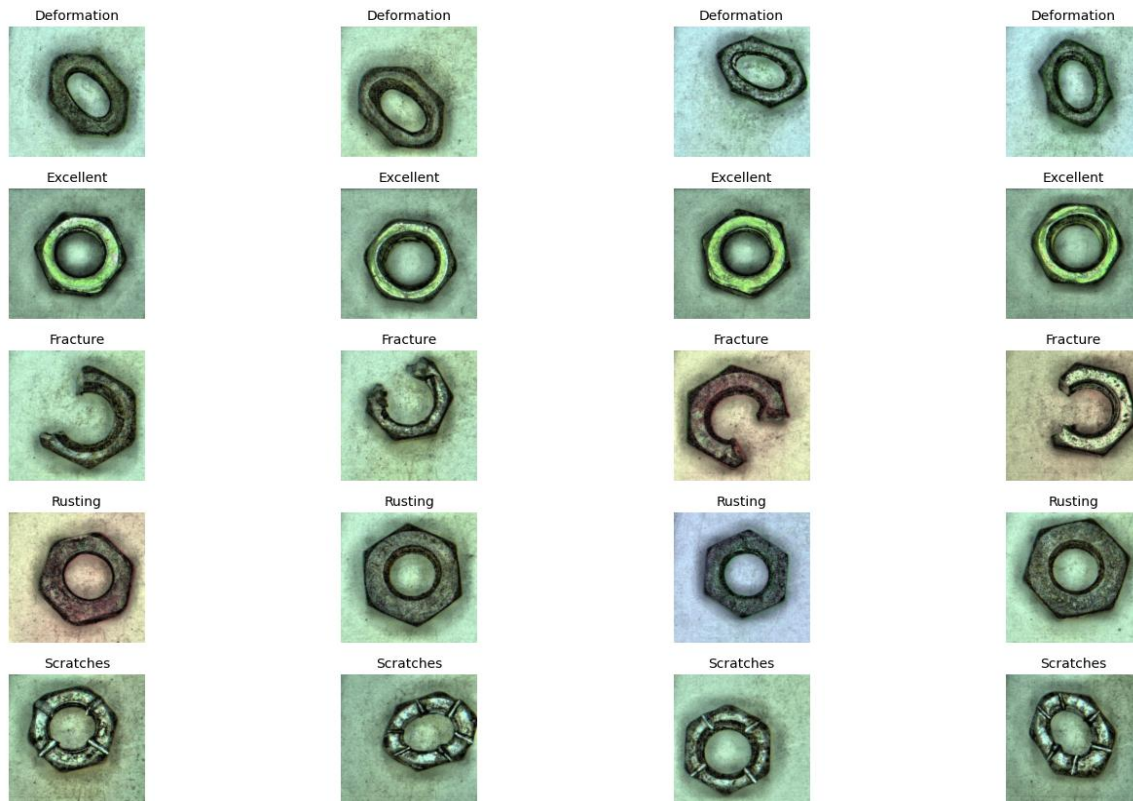
The model uses ImageNet pretrained weights. The feature extraction layers of the EfficientNet-B0 were initially frozen. The original classifier head was replaced with a custom sequential block comprising:

- A Dropout layer ($p=0.2$) to mitigate overfitting.
- A fully connected (Linear) layer mapping the 1280-dimensional feature vector to 5 output classes.

3. Dataset and Preprocessing

The dataset is structured into five folders within the training directory, corresponding to the five classes.

Sample Images from Dataset



3.1 Class Distribution

The dataset is split into training and validation sets. The distribution of images across the classes is as follows:



Class	Training Images	Validation Images	Category
Deformation	1471	368	Defective
Excellent	249	63	Non-Defective
Fracture	229	58	Defective
Rusting	423	106	Defective
Scratches	378	95	Defective

The training set contains 2750 samples, while the validation set contains 690 samples. Due to the class imbalance (e.g., Deformation comprises 53.49% of the training data), normalized class weights were calculated and applied to the Cross-Entropy Loss function to penalize misclassifications of minority classes heavily.

3.2 Data Preprocessing and Augmentation

To enhance the model's generalization capabilities, the training images undergo a robust augmentation pipeline:

- Random Resized Crop to 224x224 pixels.
- Random Horizontal and Vertical Flips.
- Random Rotation up to 15 degrees.
- Color Jittering (brightness, contrast, saturation, and hue).
- Normalization using standard ImageNet mean [0.485, 0.456, 0.406] and standard deviation [0.229, 0.224, 0.225].

Validation images are resized to 256x256, center-cropped to 224x224, and normalized. The data is divided into batches of 32, resulting in 86 batches for training and 22 batches for validation.

4. Training Pipeline and Methodology

The training was conducted in two distinct stages to preserve the pre-trained weights while adapting the model to the specific dataset.

4.1 Stage 1: Classifier Training

In the first stage, the EfficientNet-B0 feature extractor was frozen, and only the custom classifier head was trained. This prevents the large, uninitialized gradients of the new classifier from distorting the pre-trained features.

- Optimizer: AdamW with a learning rate of 1e-3.
- Epochs: 10
- Scheduler: ReduceLROnPlateau

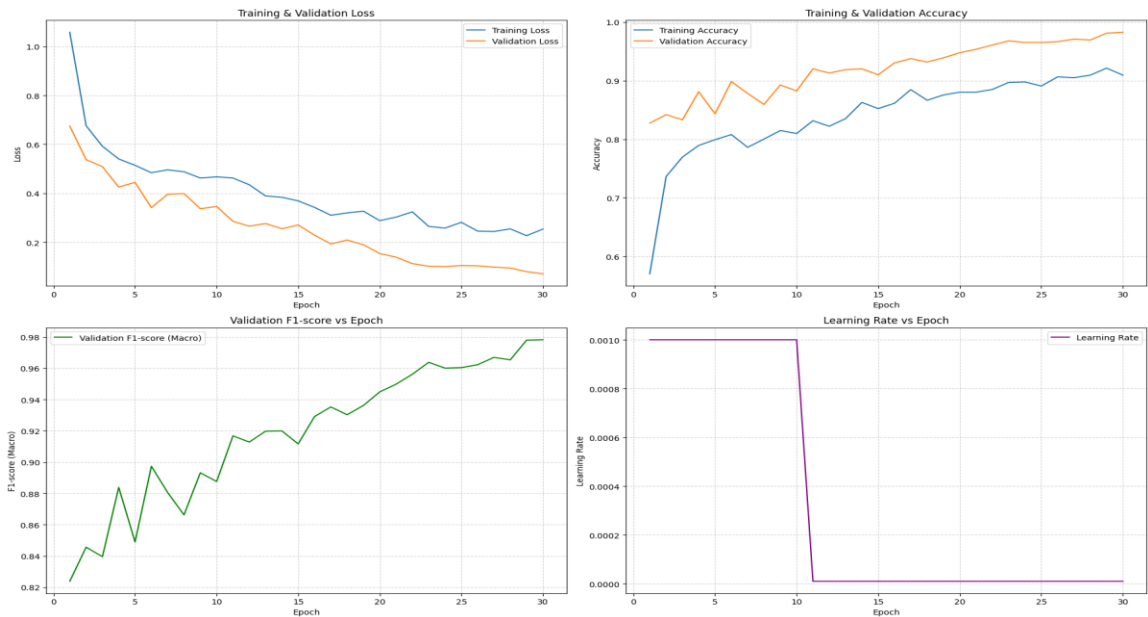
4.2 Stage 2: Fine-Tuning the Full Network

In the second stage, the entire network was unfrozen to allow fine-tuning of the feature maps specifically for nut surface defects.

- Optimizer: AdamW with a lower learning rate of 1e-5.
- Epochs: 20
- Scheduler: ReduceLROnPlateau

4.3 Epoch-wise Training and Validation Metrics

The following table summarizes the loss, accuracy, and macro F1-score across the training stages.



Stage	Epoch	Train Loss	Train Acc	Train F1	Val Loss	Val Acc	Val F1
1	1	1.0579	0.5702	0.5673	0.6745	0.8275	0.8238
1	2	0.6761	0.7364	0.7199	0.5374	0.8420	0.8456
1	3	0.5921	0.7695	0.7505	0.5089	0.8333	0.8395
1	4	0.5401	0.7895	0.7719	0.4253	0.8812	0.8838
1	5	0.5143	0.7993	0.7765	0.4445	0.8435	0.8489
1	6	0.4840	0.8080	0.7869	0.3412	0.8986	0.8973
1	7	0.4959	0.7862	0.7656	0.3965	0.8783	0.8805
1	8	0.4878	0.8004	0.7775	0.3988	0.8594	0.8662
1	9	0.4622	0.8149	0.7934	0.3370	0.8928	0.8932
1	10	0.4672	0.8098	0.7912	0.3461	0.8826	0.8876
2	1	0.4623	0.8316	0.8071	0.2855	0.9203	0.9168
2	2	0.4347	0.8222	0.7998	0.2655	0.9130	0.9128
2	3	0.3892	0.8353	0.8174	0.2764	0.9188	0.9197
2	4	0.3835	0.8629	0.8416	0.2551	0.9203	0.9199
2	5	0.3690	0.8524	0.8337	0.2704	0.9101	0.9116
2	6	0.3422	0.8615	0.8427	0.2289	0.9304	0.9291
2	7	0.3102	0.8847	0.8647	0.1931	0.9377	0.9353
2	8	0.3198	0.8665	0.8493	0.2088	0.9319	0.9303
2	9	0.3269	0.8756	0.8566	0.1897	0.9391	0.9362
2	10	0.2881	0.8804	0.8612	0.1534	0.9478	0.9449
2	11	0.3027	0.8804	0.8622	0.1388	0.9536	0.9498
2	12	0.3240	0.8851	0.8610	0.1126	0.9609	0.9562
2	13	0.2645	0.8971	0.8783	0.1018	0.9681	0.9638
2	14	0.2576	0.8978	0.8794	0.1004	0.9652	0.9601
2	15	0.2813	0.8909	0.8695	0.1047	0.9652	0.9604
2	16	0.2452	0.9065	0.8900	0.1036	0.9667	0.9622
2	17	0.2439	0.9051	0.8867	0.0982	0.9710	0.9669
2	18	0.2544	0.9095	0.8917	0.0945	0.9696	0.9653
2	19	0.2267	0.9215	0.9029	0.0797	0.9812	0.9780
2	20	0.2536	0.9095	0.8893	0.0708	0.9826	0.9782

5. Evaluation Metrics and Validation

To validate the model, several statistical metrics were utilized: Precision, Recall, and F1-score.

- Precision: Measures the accuracy of positive predictions (minimizing false positives).
- Recall: Measures the ability to find all positive instances (minimizing false negatives).
- F1-Score: The harmonic mean of precision and recall, providing a balanced metric, especially for imbalanced datasets.
- Support: The actual number of occurrences of the class in the specified dataset.

5.1 Final Validation Metrics

The final model achieved a Validation Accuracy of 98.41% and a Macro F1-score of 0.9798.

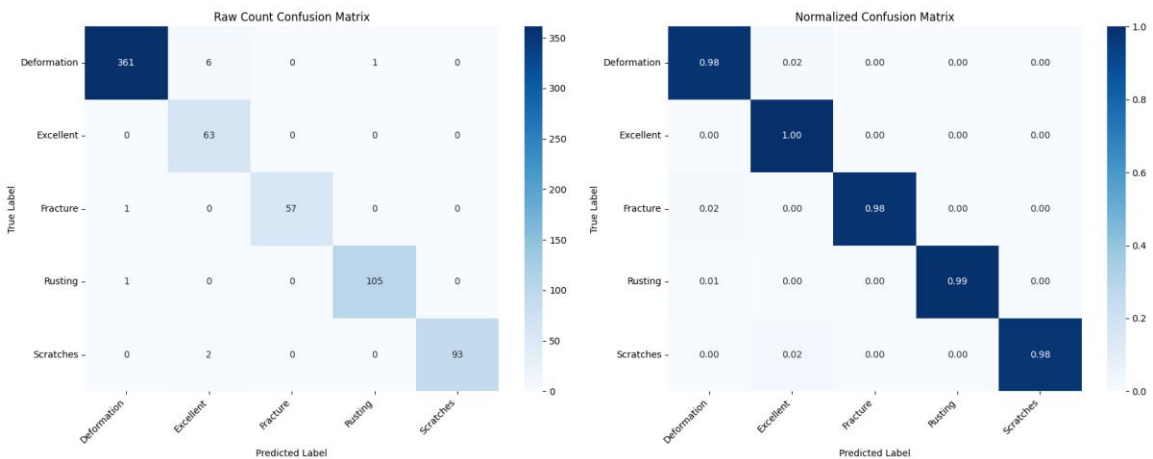
Class	Precision	Recall	F1-Score	Support
-------	-----------	--------	----------	---------

Deformation	0.9945	0.9810	0.9877	368
Excellent	0.8873	1.0000	0.9403	63
Fracture	1.0000	0.9828	0.9913	58
Rusting	0.9906	0.9906	0.9906	106
Scratches	1.0000	0.9789	0.9894	95

5.2 Confusion Matrix Analysis

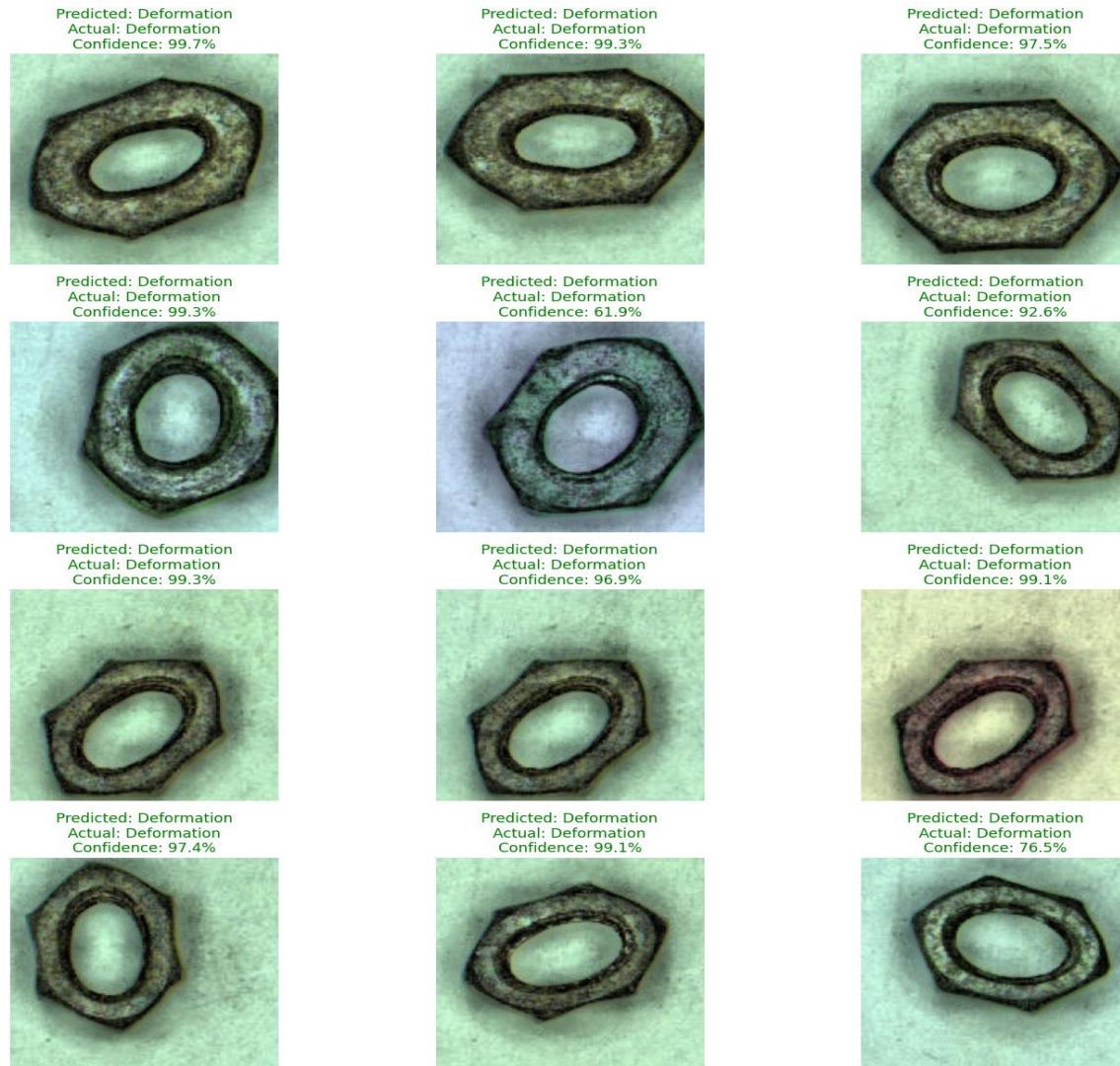
A confusion matrix is a critical evaluation tool that visualizes the performance of an algorithm. It displays the true labels against the predicted labels, allowing us to identify exactly where the model is confusing two classes. This is vital for industrial applications where classifying a defective part as 'Excellent' (False Positive) could result in faulty products reaching the consumer.

The confusion matrix for our model demonstrated exceptional performance, with the most frequent confusion being 'Scratches' predicted as 'Excellent' occurring in only 2.11% of Scratches images. We know the model is performing well because the diagonal values (True Positives) are extremely high compared to the off-diagonal values (misclassifications).



6. Model Predictions and Inference

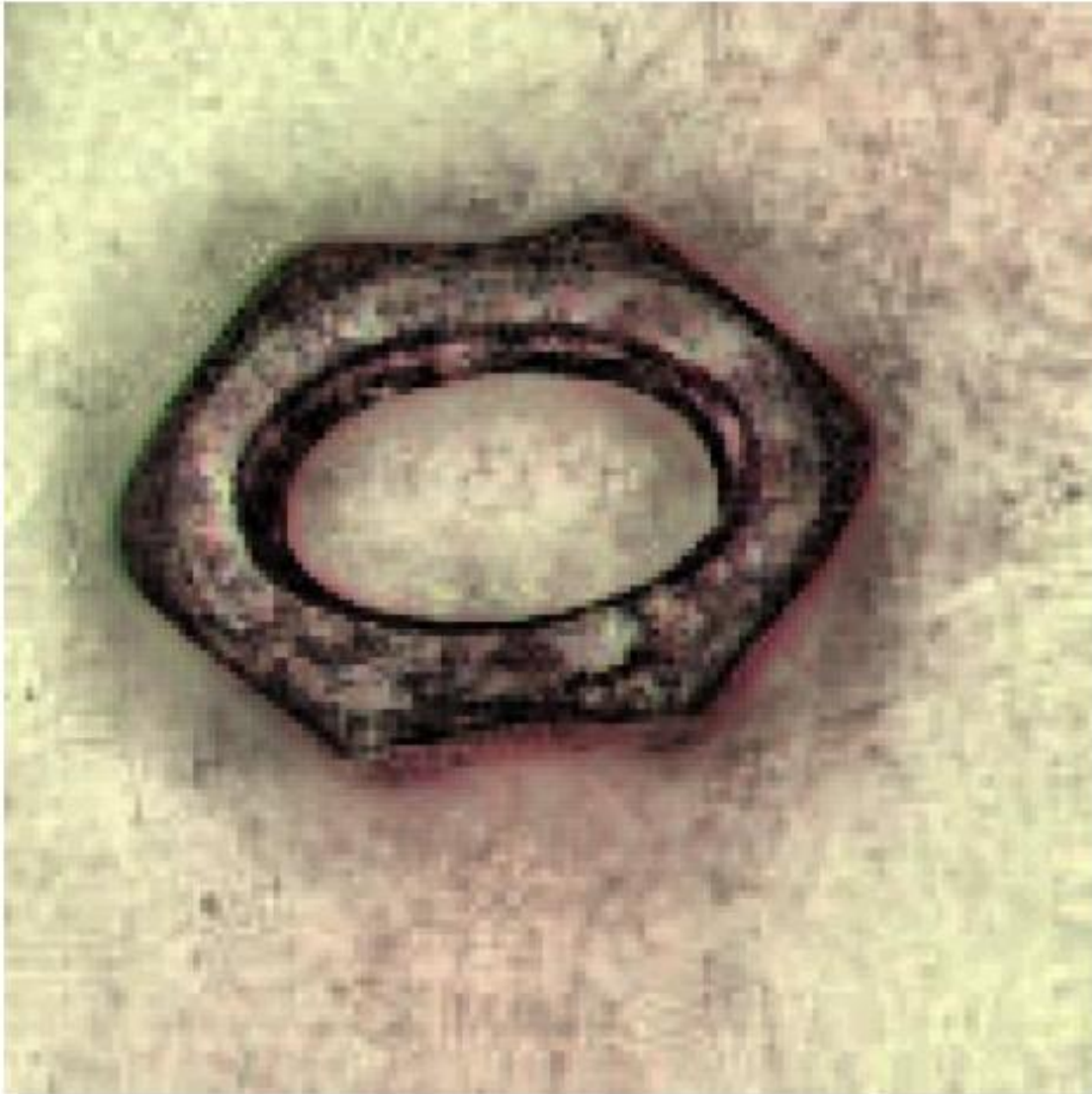
During evaluation, we classify predictions into correct and incorrect categories to qualitatively assess the model's behavior. Correct predictions feature high confidence scores (often >99%) across distinct defect types. Incorrect predictions help us understand edge cases, such as when minor scratches are interpreted as normal surface variations.



6.1 Single Image Prediction Function

To deploy this model in a real-world manufacturing pipeline, a single-line prediction function was developed. This function takes a raw image from a camera sensor, applies the exact validation transformations (resizing, center cropping, normalization), passes the tensor through the loaded model in evaluation mode (``torch.no_grad()``), and extracts the softmax probabilities to determine the top predicted class and its confidence. This function is essential because it bridges the gap between batch-level training and real-time, item-by-item inference required on a factory conveyor belt.

Prediction: Deformation
Confidence: 90.62%



7. Conclusion

The proposed EfficientNet-B0 based framework successfully automates the defect detection process for manufactured parts. By reframing the multi-class problem into an overarching defective vs. non-defective paradigm, product-based companies can seamlessly integrate this lightweight, highly accurate model into their production lines to eliminate manual classification.

References

1. Weihaoreal. (n.d.). Nut Surface Defect Dataset. Kaggle. Retrieved from <https://www.kaggle.com/datasets/weihaoreal/nut-surface-defect-dataset>
2. Wu, D., Hong, Y., Wang, J., Wu, S., Zhang, Z., & Liu, Y. (2025). EfficientNet-b0-Based 3D Quantification Algorithm for Rectangular Defects in Pipelines. IEEE Access.
3. Shin, M., Seo, J., Lee, I.-B., & Kim, S. (2026). Defective Photovoltaic Module Detection Using EfficientNet-B0 in the Machine Vision Environment. Machines, MDPI.